Next-Day Rain Classification for the Colombo Area

Machine Learning Assignment

MSc in Artificial Intelligence
University of Moratuwa

Pansilu Wijesiri
269220E
October 2026

Abstract

This study uses recent weather and the time of year to classify the following day as wet or dry in the Colombo area. The compiled dataset contains 4,383 daily ECMWF ERA5 records obtained through Open-Meteo for 2014-2025. A wet day was defined as at least 1 mm of precipitation. After preparing weather-history features, 4,376 rows remained. Regularised Quadratic Discriminant Analysis (QDA) was trained on 2014-2021, selected using 2022-2023 and evaluated on 2024-2025. Test ROC AUC was 0.873, balanced accuracy 0.786 and F1 0.873.

QDA was chosen as a non-deep-learning algorithm outside the supplied lecture list. It did not outperform every baseline: logistic regression gave better probability ranking and lower Brier loss, while rain persistence gave higher accuracy and F1. Exact interventional SHAP explains each prediction in a Streamlit application. The main limitation is that ERA5 is delayed reanalysis; the results describe historical classification and do not establish real-time forecasting skill.

1. Problem and dataset

Daily rain affects transport, outdoor work and ordinary planning in Colombo. The problem here is deliberately narrow: using information up to day t, classify whether precipitation on day t+1 reaches 1 mm. The target does not describe rainfall severity, timing within the day or flood risk. This makes the task suitable for a small tabular classification model.

The data were downloaded on 27 October 2026 from the Open-Meteo Historical Weather API, explicitly selecting ECMWF ERA5 [1]. The extract covers 1 January 2014 to 31 December 2025, with daily aggregation in Asia/Colombo. The request used 6.9271 N, 79.8612 E; the API returned the grid location 7.0 N, 80.0 E. This spatial difference matters: the data represent a Colombo-area grid cell rather than a measurement at a particular street or weather station.

Input	Description
Preceding-day rain	Daily precipitation, mm
Humidity	Daily mean relative humidity, %
Maximum / minimum temperature	Daily maximum and minimum air temperature, C
Wind	Daily maximum 10 m wind speed, km/h
Pressure	Daily mean surface pressure, hPa
3-day / 7-day mean rain	Trailing means ending on the preceding day, mm/day
Pressure change	Preceding-day pressure minus the day before, hPa
Season sine / cosine	Two values derived from the target date

Data checks and preparation

All dates were unique and consecutive, and the six raw weather variables had no missing values. Rainfall was checked for negative values and minimum temperature against maximum temperature. No imputation or outlier removal was needed. The first six days were removed because a full seven-day history was unavailable. The final day was removed because its next-day target was outside the extract. This left 4,376 supervised rows and 11 numerical predictors.

The original JSON, daily CSV and derived dataset are included with the source. The exact request, units, collection time and raw SHA-256 are recorded in data/provenance.json. The data contain no personal information. Open-Meteo and ECMWF ERA5 are credited, and the conversion and derived predictors are identified under CC BY 4.0 [2].

2. Methodology

2.1 Chronological split

Rows were split by target date. A random split would mix neighbouring days and similar seasonal conditions across training and testing, so it would be less useful for this task.

Partition	Target dates	Rows	Wet days
Train	2014-01-08 to 2021-12-31	2915	2116
Validation	2022-01-01 to 2023-12-31	730	572
Test	2024-01-01 to 2025-12-31	731	556

Rainfall history and pressure changes use day t and earlier days only. The target is precipitation on t+1, retained for evaluation but excluded from the inputs. The known calendar date of t+1 supplies sine and cosine season features, using angle 2*pi*(day-of-year - 1)/365.25. These avoid treating December and January as unrelated ends of a numerical scale.

2.2 Algorithm selection

The supplied lecture list includes linear and logistic regression, decision trees, random forest, SVM, KNN, XGBoost, AdaBoost, gradient boosting, LightGBM and CatBoost. QDA is outside that list and uses no neural network or deep learning. Logistic regression is retained only as a comparison baseline, not as the selected assignment algorithm.

QDA estimates a multivariate Gaussian density for each class, with separate means and covariance matrices, then applies Bayes' rule [3]. Unequal covariances produce a quadratic decision boundary. This gives a compact way to model interactions between weather variables without constructing a tree ensemble. Regularisation stabilises covariance estimates when rain-history inputs are correlated.

Unlike logistic regression, QDA models the distribution of inputs within each class rather than fitting one linear log-odds function directly. Decision trees divide the space using sequential rules; QDA uses continuous density estimates. KNN compares distances to stored examples; QDA summarises each class through fitted parameters. Its Gaussian assumptions are a trade-off, especially for rainfall.

3. Training and evaluation procedure

The three rainfall inputs were transformed with log(1 + rainfall) to reduce positive skew and retain zero rain. All 11 inputs were then standardised using means and standard deviations fitted on training rows only. The transformation was fixed before validation selection. Class priors were the training wet/dry frequencies; there was no class resampling.

Eight covariance regularisation settings were compared on validation AUC. In standardised coordinates, each class covariance is shrunk towards the identity: S_regularised = (1 - alpha)*S + alpha*I. QDA was fitted using scikit-learn, and the selected model was exported as a quadratic score with the same preprocessing statistics [3].

Regularisation alpha	Validation AUC
0.00	0.8846
0.01	0.8854
0.05	0.8874
0.10	0.8890
0.25	0.8924
0.50	0.8960
0.75	0.8978
1.00	0.8972

Alpha 0.75 gave the highest validation AUC, 0.8978. A threshold grid from 0.20 to 0.80 in steps of 0.005 selected 0.78 using validation balanced accuracy. The model was not refitted on validation or test rows. This cutoff weights wet- and dry-day recall equally; it was not derived from measured real-world error costs.

Metrics and baselines

ROC AUC measures ranking across thresholds. Balanced accuracy averages wet-day and dry-day recall. F1 combines wet-day precision and recall. Brier loss measures squared probability error, with lower values better. These supplement accuracy because 556 of the 731 test days were wet: predicting wet throughout would already give 76.06% accuracy.

Baselines were logistic regression with train-only standardisation, training-month wet-day frequencies, preceding-day rain persistence and constant training prevalence. Logistic and monthly thresholds were selected on validation rows. Approximate AUC intervals use 1,000 moving-block bootstrap samples with 30-day blocks, preserving some short-term dependence. Seed 269220 fixes the bootstrap; QDA fitting is deterministic.

4. Results

Model	AUC	Accuracy	Bal. acc.	F1	Brier
Regularised QDA	0.873	0.814	0.786	0.873	0.132
Logistic regression	0.885	0.807	0.801	0.865	0.107
Monthly climatology	0.825	0.811	0.807	0.868	0.131
Rain persistence	0.775	0.836	0.775	0.892	0.164
Constant train prevalence	0.500	0.761	0.500	0.864	0.183

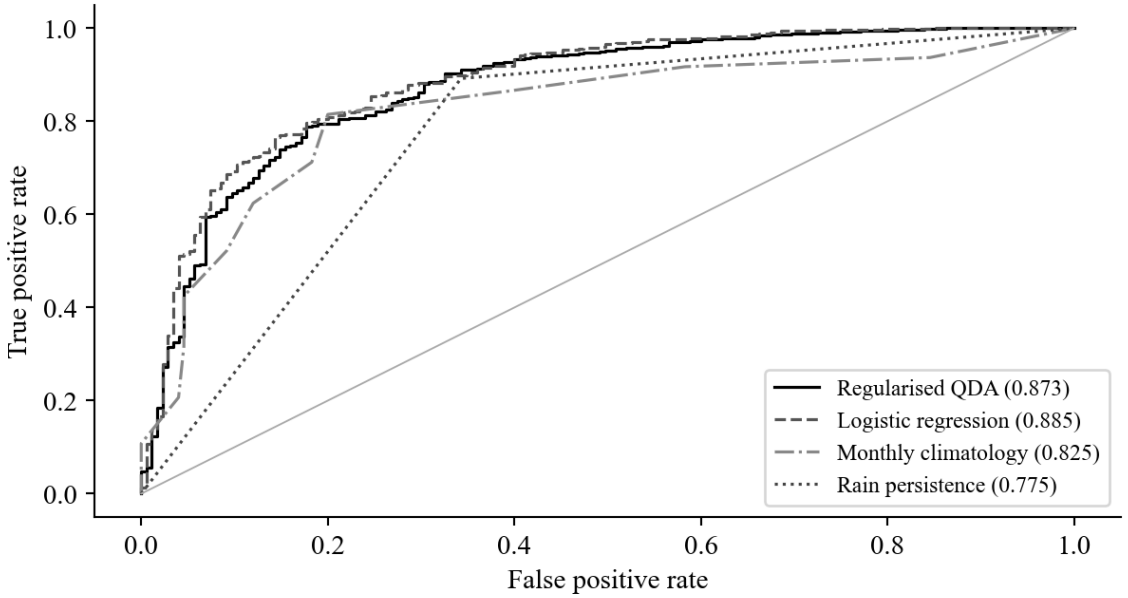


Figure 1. ROC curves for the held-out 2024-2025 period. The diagonal represents random ranking.

QDA achieved an AUC of 0.873, compared with 0.885 for logistic regression and 0.825 for monthly climatology. Its Brier loss of 0.132 was worse than logistic regression (0.107) and slightly worse than monthly climatology (0.131). Persistence gave higher accuracy and F1, while monthly climatology gave higher balanced accuracy. QDA is therefore a useful study model, but the results do not justify calling it the strongest model overall.

QDA average precision was 0.946. The approximate 95% AUC interval was 0.806-0.916. The paired AUC difference against monthly climatology had an interval of 0.005-0.081. These intervals describe a fixed historical period; they do not establish performance at a new site or with real-time forecast inputs.

5. Errors and explainability

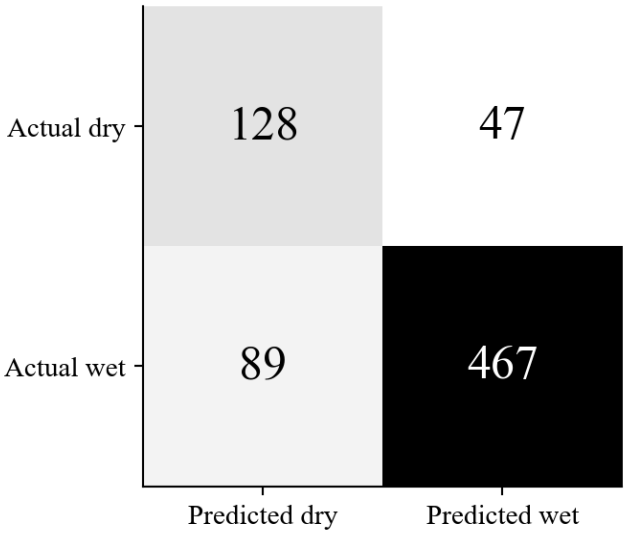


Figure 2. Test confusion matrix at the validation-selected threshold of 0.78.

The model correctly classified 467 wet days and 128 dry days. It missed 89 wet days and produced 47 false wet alerts. Wet-day recall was 83.99%, dry-day recall 73.14% and wet-day precision 90.86%. A user mainly concerned about missed rain might prefer a lower threshold, but that would create more false alerts and needs a separate cost-based decision.

Exact interventional SHAP

SHAP assigns contributions to features relative to a background [4]. Here the explained function is the QDA raw log-odds score: $f(z) = c + b'z + z'Az$, where z contains the transformed, standardised inputs. The empirical distribution of all 2,915 training rows is the background. Linear and squared terms belong to their feature; pairwise terms are allocated using the Shapley rule. This gives exact interventional contributions without approximation by sampled permutations.

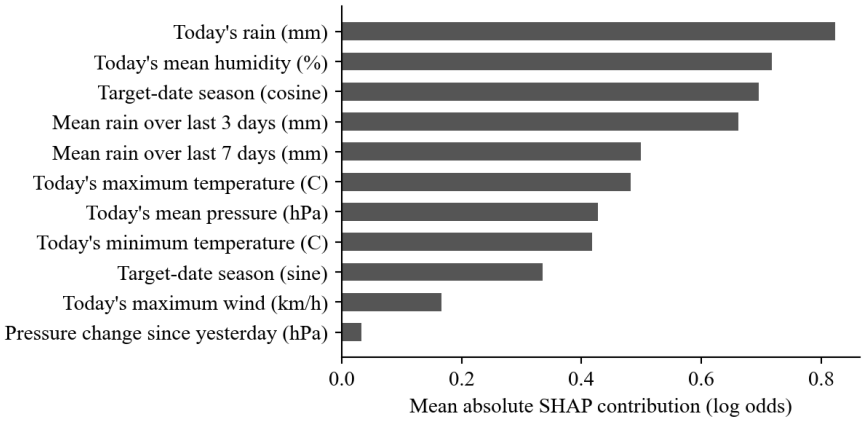


Figure 3. Mean absolute interventional SHAP contributions across 240 evenly spaced held-out days.

Preceding-day rain had the largest mean absolute contribution (0.823 log odds), followed by humidity (0.718) and cosine season (0.696). Recent wet conditions and seasonality are plausible rain-day associations. Replacement coalitions can break relationships between overlapping rain windows and the season coordinates; individual credit is not a causal effect.

6. Interpreting individual predictions

Example	Date	Model p(wet)	Actual rain
Correct wet day	2025-01-01	90.5%	9.4 mm
Correct dry day	2025-01-04	29.6%	0.0 mm
Missed wet day	2024-12-19	74.9%	2.0 mm
False wet alert	2025-01-20	99.9%	0.7 mm

For 2025-01-01, preceding-day rain was 6.6 mm and mean humidity was 85%. The model gave a 90.53% wet-day probability. The largest upward contribution was Today's mean humidity (%) (+0.568 log odds); the largest downward contribution was Target-date season (cosine) (-1.020). The base score 2.273 plus all 11 contributions gave 2.257. A logistic transform gives the displayed probability. Actual next-day rain was 9.4 mm.

The missed wet example received a 74.9% score, below the 78% cutoff, although 2.0 mm fell. The false alert received 99.9% but only 0.7 mm fell. This near-boundary rainfall and very confident wrong prediction show the limits of a binary target and Gaussian class densities. An explanation describes the fitted model's score; it does not establish that the prediction is correct.

Probability quality

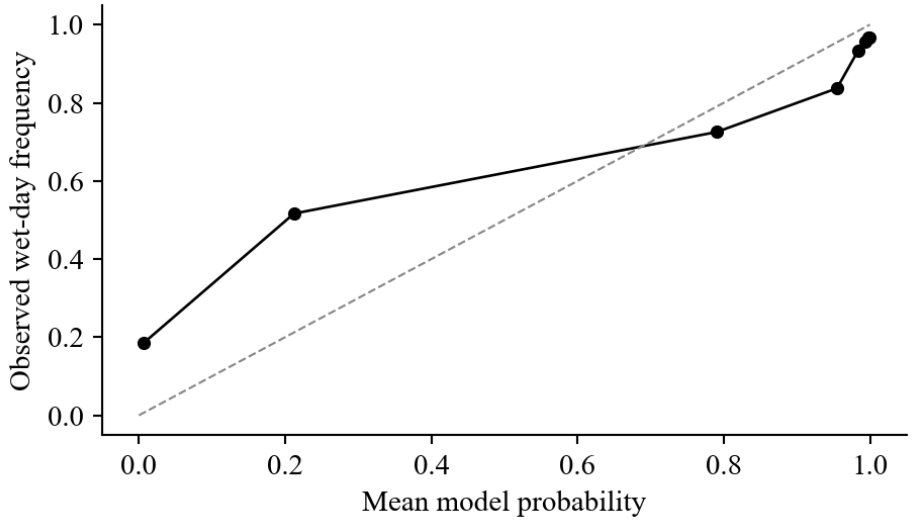


Figure 4. Eight quantile bins compare model probabilities with observed wet-day frequency.

No separate calibration model was fitted. The descriptive calibration plot and relatively high Brier loss caution against treating an individual score as a reliable frequency. The SHAP baseline is mean training log odds, not mean training probability. Contributions reconstruct the score, with maximum error below 4e-15 across the 240 explained test days. They are model associations rather than causal weather effects.

7. Streamlit application

The Python application loads models/qda.json once and accepts nine weather-history quantities and a target date. The date supplies two season features. The same feature order, log-rain transformation and training-only standardisation are used for prediction. The JSON stores fitted score coefficients and training-background moments; the interface does not retrain the model.

The first tab follows a weather-notebook layout, with inputs on one side and the result and explanation on the other. Four held-out examples show correct wet/dry decisions and both error types. Users can change values and calculate a new result. Exact interventional SHAP is computed for every submitted entry, so the explanation corresponds to the actual input.

Validation and reproducibility

The form checks numerical limits, minimum/maximum temperature consistency and whether rolling rain totals include the shorter windows. It flags values outside the training range. Streamlit AppTest checks example switching and invalid entries. The shared input builder reconstructs all 731 held-out feature rows and reproduces their saved probabilities.

Twelve automated checks cover data alignment, source checksum, preprocessing, inference and the application. The portable QDA score is independently compared with a fitted scikit-learn QDA model on validation and test rows. A separate explanation check enumerates all 2,048 feature coalitions against 32 background rows, calculates their exact Shapley-weighted effects and compares them with the analytic implementation. This checks individual contributions as well as score additivity.

The results tab displays the baseline comparison, confusion matrix, global SHAP and validation search. The data tab explains the source, previews rows and offers a CSV download. The interface identifies the output as a historical research demonstration rather than a live forecast.

Running and deployment

The entry point is streamlit_app.py. Install requirements.txt and run python -m streamlit run streamlit_app.py. Streamlit Community Cloud can deploy the same entry point from GitHub, using Python 3.12 and the main branch [5]. No API key or separate prediction service is needed. Local checks establish application behaviour; a hosted deployment still needs an access and visual check.

8. Discussion and conclusion

Data and model limitations

ERA5 is retrospective reanalysis. Even though features stop before the target day, complete reanalysis inputs would not necessarily have been available when a real next-day forecast was issued. A chronological split prevents target leakage but does not remove that delay. An operational study would need inputs available at each issue time.

The returned grid point differs from the requested Colombo coordinates and cannot capture every coastal or neighbourhood shower. A local station comparison would help assess how well the target represents experienced rain. The target itself loses information: 0.7 mm and 1.7 mm fall in different classes despite both being small amounts.

QDA assumes Gaussian class densities. Rainfall has many small values and a long upper tail, and log transformation cannot remove all departures from that assumption. The seasonal coordinates lie on a circle, and rolling rain means overlap. Covariance regularisation reduces instability but does not make these assumptions true. Logistic regression's stronger test AUC and Brier loss show that additional flexibility did not automatically improve generalisation.

Bias, uncertainty and impact

Wet days dominate the extract. Some test months contain very few or no dry days, so aggregate scores can hide seasonal weaknesses. The model contains no personal attributes, but a single-city model offers geographically uneven coverage. False confidence could affect people working outdoors. The application therefore does not give safety or flood advice.

Validation rows were reused for hyperparameter and threshold selection, which can make validation scores optimistic. The final test contains only two years. Bootstrap intervals depend on block length and do not fully capture long seasonal dependence. Algorithm eligibility was established from the supplied lecture list; the regularisation and cutoff were selected using validation data, rather than test performance.

Conclusion and next steps

Regularised QDA met the algorithm-selection constraint and captured useful historical wet-day patterns, with test AUC 0.873. The baseline comparisons show where it was weaker, especially probability quality. SHAP and the Streamlit frontend make its behaviour inspectable. Station validation, more locations, rolling-origin evaluation, time-available inputs and independent calibration would be needed before practical forecasting use.

9. Reproducing the work

The source contains the fixed daily data, provenance, derived dataset, training/evaluation scripts, saved QDA model, test predictions, SHAP outputs and Streamlit entry point. Package versions are pinned in requirements.txt. The experiment uses Python 3.12. Recollecting data may obtain an upstream revision, so retain the included snapshot and checksum for direct comparisons.

Command	Purpose
python -m src.preprocess	Prepare the modelling dataset
python -m src.train	Train, select, evaluate and explain QDA
python -m src.evaluate	Check frozen test metrics
python -m src.explain	Recompute SHAP and check additivity
python -m unittest discover -s tests -p 'test_*.py'	Check data, model, XAI and interface
python -m streamlit run streamlit_app.py	Start the application

References

[1] Open-Meteo. Historical Weather API documentation.

https://open-meteo.com/en/docs/historical-weather-api

[2] Open-Meteo. Data licence: Creative Commons Attribution 4.0.

https://open-meteo.com/en/licence

[3] Scikit-learn. QuadraticDiscriminantAnalysis, version 1.8 documentation.

scikit-learn.org / QuadraticDiscriminantAnalysis / version 1.8

[4] Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems, 30.

https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions

[5] Streamlit. Deploy your app on Community Cloud.

https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy

Online sources accessed 27 October 2026. The scope follows the supplied Machine Learning Assignment - Guidelines and the supplied lecture-algorithm list.